

LC 986: Interval List Intersection

Pinterest

Snapchat

★★★

© Authored by: Arpit Raj, LNMIIT Jaipur | Date: July 15, 2026

Problem Statement

Given 2 lists in sorted order, we need to return their intersection. Individually, they are disjoint.

Core Intuition: Whenever we see 2 sorted arrays, process both simultaneously and advance one pointer based on some condition.

Intersection Logic

When processing current intervals from both arrays (let's say A and B), we can find the potential intersection boundaries as follows:

```
start = max(A.start, B.start)
end = min(A.end, B.end)
```

We have a valid intersection **when** `start <= end`.

Pointer Movement Strategy

Only one pointer moves in every iteration. Now, to decide which pointer moves:

Example:

A = [1, 5]

B = [2, 100]

A ends first, hence (A++) as it can intersect B again.

Condition:

```
if (A[i][1] < B[j][1])  
    i++  
else  
    j++
```

Template Structure:

```
int i = 0;  
int j = 0;  
while (i < n && j < m) {  
    // process current pair  
    if (condition) {  
        i++;  
    } else {  
        j++;  
    }  
}
```

Brute Force Approach

The brute force approach would have $O(n*m)$ time complexity, as it would check all start and end positions and push to ans.

```
for (auto &a : A) {  
    for (auto &b : B) {  
        int s = max(a[0], b[0]);  
        int e = min(a[1], b[1]);  
        if (s <= e) {  
            ans.push({s, e});  
        }  
    }  
}
```

Time Complexity: $O(n * m)$

Space Complexity: $O(1)$

Optimized C++ Solution

Time Complexity: $O(n + m)$

Space Complexity: $O(1)$

```

class Solution {
public:
    vector<vector<int>> intervalIntersection(vector<vector<int>>& list1, vector<vect

        int n = list1.size();
        int m = list2.size();

        int i = 0;
        int j = 0;

        vector<vector<int>> ans;
        while(i < n && j < m) {
            int start = max(list1[i][0], list2[j][0]);
            int end = min(list1[i][1], list2[j][1]);

            if(start <= end) {
                ans.push_back({start, end});
            }

            if(list1[i][1] < list2[j][1]) { // agar list1 ka element pehle khtm ho i
                i++;
            }
            else {
                j++;
            }
        }

        return ans;
    }
};

```